

This is unreleased documentation for Serverpod **Next** version.

For up-to-date documentation, see the [latest version](#) (4.0.0).

3..20..19

CONFIGURATION

 Copy as Markdown

Configure Firebase authentication

This page covers additional configuration options for the Firebase identity provider beyond the basic setup.

Loading credentials with FirebaseAuthConfig

The [setup guide](#) uses `FirebaseIdpConfigFromPasswords`, which loads the service account key from `passwords.yaml` for you. When you need to load credentials from a different source (a file path, a secrets manager, or just a project ID), use `FirebaseIdpConfig` directly and pass a `FirebaseServiceAccountCredentials` instance.

The `FirebaseServiceAccountCredentials` class provides four constructors. These are the only supported ways to construct it:

From a JSON string (use this when reading the JSON from a secrets manager or environment variable):

```
final firebaseIdpConfig = FirebaseAuthConfig(  
  credentials: FirebaseServiceAccountCredentials.fromJsonString(  
    pod.getPassword('firebaseServiceAccountKey')!,  
  ),  
);
```

From a JSON file (useful for local development or when secrets are mounted as files):

```
import 'dart:io';  
  
final firebaseIdpConfig = FirebaseAuthConfig(  
  credentials: FirebaseServiceAccountCredentials.fromJsonFile(  
    File('config/firebase_service_account_key.json'),  
  ),  
);
```



From a JSON map (useful when credentials are assembled programmatically, for example by pulling each field from `passwords.yaml` or a secrets manager):

```
final firebaseIdpConfig = FirebaseIdpConfig(
  credentials: FirebaseServiceAccountCredentials.fromJson({
    'type': 'service_account',
    'project_id': pod.getPassword('firebaseProjectId')!,
    'private_key_id': pod.getPassword('firebasePrivateKeyId')!,
    'private_key': pod.getPassword('firebasePrivateKey')!,
    'client_email': pod.getPassword('firebaseClientEmail')!,
    'client_id': pod.getPassword('firebaseClientId')!,
    'auth_uri': 'https://accounts.google.com/o/oauth2/auth',
    'token_uri': 'https://oauth2.googleapis.com/token',
  }),
);
```

WARNING

Do not inline the service account fields (especially `private_key`) directly in source code. Load every sensitive field from a secure source such as `pod.getPassword()` (backed by `passwords.yaml` or `SERVERPOD_PASSWORD_*` environment variables) or a secrets manager.

Project ID only (Serverpod uses only this field and ignores the rest):

```
final firebaseIdpConfig = FirebaseIdpConfig(
  credentials: const FirebaseServiceAccountCredentials(
    projectId: 'your-project-id',
  ),
);
```

NOTE

Only `projectId` is used from the service account JSON. The other fields are accepted so you can paste the downloaded file unchanged, but Serverpod does not use them. ID token signatures are verified against Google's public certificates, not against the service account key.

Custom account validation

You can customize the validation for Firebase account details before allowing sign-in. By default every account is accepted, including one whose email is not verified. Firebase Email/Password accounts start unverified and users usually sign in straight after signing up, so rejecting them would block that flow.

To require a verified email instead, pass the built-in validator:

```
final firebaseIdpConfig = FirebaseAuthConfigFromPasswords(  
    firebaseAccountDetailsValidation: FirebaseAuthConfig.requireVerifiedEmail,  
);
```

It throws `FirebaseEmailNotVerifiedException`, which reaches the app so you can prompt the user to verify. Accounts with no email, such as phone sign-in, are still accepted.

To customize validation, provide your own `firebaseAccountDetailsValidation` function:

```
final firebaseIdpConfig = FirebaseAuthConfigFromPasswords(  
    firebaseAccountDetailsValidation: (accountDetails) {  
        // Require verified email (even for phone auth). Throw the serializable  
        // FirebaseEmailNotVerifiedException so the app can tell this case apart.  
        // A plain Exception reaches the app as FirebaseAuthTokenVerificationException.  
        if (accountDetails.verifiedEmail != true) {  
            throw FirebaseEmailNotVerifiedException();  
        }  
  
        // Restrict to specific email domain  
        if (accountDetails.email != null &&  
            !accountDetails.email!.endsWith('@example.com')) {  
            throw Exception('Only @example.com emails allowed');  
        }  
    },  
);
```

FirebaseAccountDetails properties

The `firebaseAccountDetailsValidation` callback receives a `FirebaseAccountDetails` record with the following properties:

- `userIdentifier` (`String`): Firebase UID.
- `email` (`String?`): Email address, or `null` for phone-only sign-in.
- `fullName` (`String?`): Display name from Firebase.
- `image` (`Uri?`): Profile image URL.
- `verifiedEmail` (`bool?`): Whether the email is verified.
- `phone` (`String?`): Phone number, only populated for phone authentication.

Which properties are populated depends on the Firebase sign-in method the user chose. For example, `phone` is only populated for phone authentication, and `email` may be `null` if the user signed in with phone only.

Reacting to auth user creation

`onBeforeAuthUserCreated` and `onAfterAuthUserCreated` are global callbacks on `AuthUsersConfig`. They fire for every identity provider, not just Firebase. See [Working with users](#) for full

details.

The core callbacks cannot see Firebase account details, so provider-specific logic, such as a scope derived from the phone number, belongs in `onAfterFirebaseAccountCreated`, which receives the `FirebaseAccount`. The example below assigns a baseline scope to every new user:

```
pod.initializeAuthServices(  
  tokenManagerBuilders: [  
    JwtConfigFromPasswords(),  
  ],  
  identityProviderBuilders: [  
    FirebaseIdpConfigFromPasswords(),  
  ],  
  authUsersConfig: AuthUsersConfig(  
    onBeforeAuthUserCreated: (  
      session,  
      scopes,  
      blocked, {  
        required transaction,  
      }) {  
        return (  
          scopes: {...scopes, Scope('user')},  
          blocked: blocked,  
        );  
      },  
    onAfterAuthUserCreated: (  
      session,  
      authUser, {  
        required transaction,  
      }) async {  
        // e.g. send a welcome email, log for analytics  
      },  
    ),  
  );
```

For the Firebase-specific hook, pass `onAfterFirebaseAccountCreated` to the provider config. It receives the created `FirebaseAccount`, so it can read the phone number or the Firebase UID:

```
FirebaseIdpConfigFromPasswords(  
  onAfterFirebaseAccountCreated:  
    (session, authUser, firebaseAccount, {required transaction}) async {  
      if (firebaseAccount.phone != null) {  
        await AuthServices.instance.authUsers.update(  
          session,  
          authUserId: authUser.id,  
          scopes: {...authUser.scopes, Scope('phone-verified')},  
          transaction: transaction,  
        );  
      }  
    },  
  ),  
)
```

⚠ WARNING

These callbacks run inside the same database transaction as the account creation. Throwing an exception inside a callback aborts the sign-up. Wrap external side-effects (email sending, analytics) in `try/catch` so a third-party outage does not block new sign-ups.

FirebaseIdpConfig parameter reference

Parameter	Type	Required	Description
<code>credentials</code>	<code>FirebaseServiceAccountCredentials</code>	Yes	Firebase service account credentials for verifying ID tokens. Can be loaded via <code>fromJsonString</code> , <code>fromJsonFile</code> , <code>fromJson</code> , or the default constructor with just <code>projectId</code> . When using <code>FirebaseIdpConfigFromPasswords</code> , this is loaded automatically from the <code>firebaseServiceAccountKey</code> key in <code>passwords.yaml</code> or the <code>SERVERPOD_PASSWORD_firebaseServiceAccountKey</code> environment variable.
<code>firebaseAccountDetailsValidation</code>	<code>FirebaseAccountDetailsValidation</code>	No	Custom validation callback for Firebase account details before allowing sign-in. By default all account details are accepted, including unverified emails. Pass <code>FirebaseIdpConfig.requireVerifiedEmail</code> to reject accounts whose email has not been verified.
<code>onAfterFirebaseAccountCreated</code>	<code>AfterFirebaseAccountCreatedFunction?</code>	No	Callback invoked after a new Firebase account has been created and linked to an auth user. Receives the session, the created <code>AuthUserModel</code> , the <code>FirebaseAccount</code> , and the active <code>Transaction</code> . Runs inside the same database transaction as account creation, so the <code>transaction</code> can be used to perform additional database operations atomically with sign-up.
<code>clockSkewTolerance</code>	<code>Duration</code>	No	Tolerance for clock skew when validating Firebase ID token timestamps. Defaults to the framework's default clock skew tolerance.